PROBLEM 4 · SOLUTION

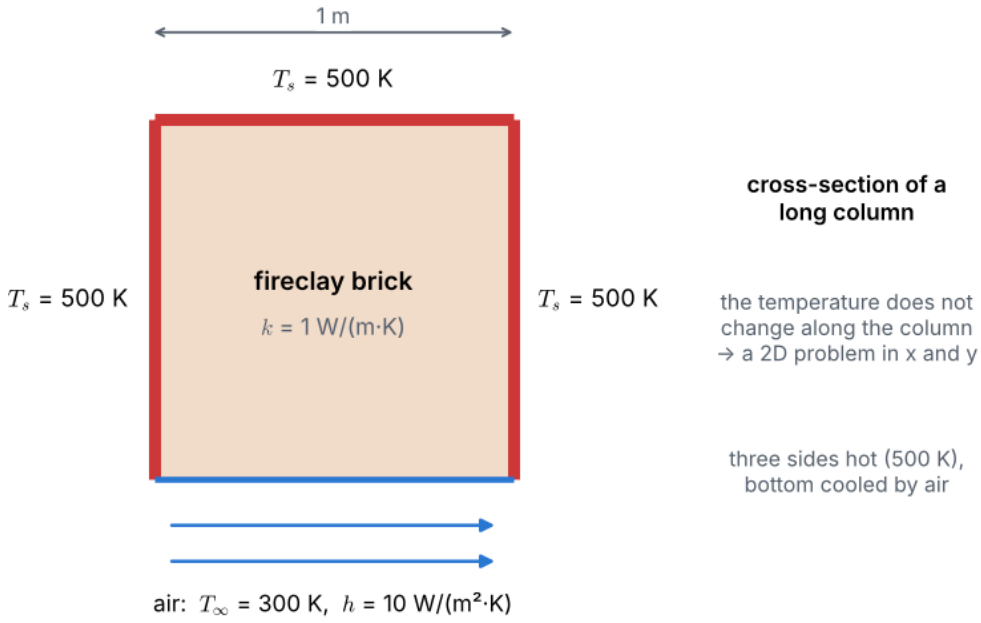
A brick column cooled by air on one side

Step-by-step solution with the Finite Difference Method

Plan. One surface now loses heat by **convection**, so its temperature is unknown and needs its own node equation. We derive every node equation with the **energy balance method** and use **symmetry** to halve the work, following the 7 pillars of CFD. All code is plain Python (lists and loops only) and is the code of the notebook `03_fdm_solver.ipynb`.

Step 1 · Geometry and data

The column is long and nothing changes along it, so the temperature varies only over the 1×1 m cross-section. Heat flows from the three 500 K surfaces through the brick to the bottom surface, where the air carries it away.



The column cross-section and its boundary conditions.

```
# ---- geometry and grid ----------------------------------------------------
W = 1.0          # width (and height) of the square cross-section [m]
dx = 0.25        # grid spacing, dx = dy                          [m]

# ---- material -------------------------------------------------------------
k = 1.0          # thermal conductivity of fireclay brick         [W/(m K)]

# ---- boundary conditions --------------------------------------------------
T_s = 500.0      # temperature of the top, left and right surfaces [K]
T_inf = 300.0    # temperature of the air stream                   [K]
h = 10.0         # convection coefficient at the bottom surface    [W/(m^2 K)]
```

Step 2 · Grid and symmetry

The 0.25 m grid has 5×5 nodes (i, j), with $j = 0$ on the bottom surface. The top, left and right nodes are known (500 K), including the two bottom corners, which belong to the hot side walls. The 9 interior nodes and the 3 bottom-surface nodes are unknown.

The geometry and boundary conditions are symmetric about $x = 0.5$ m, so column $i = 3$ has the same temperatures as column $i = 1$. That leaves **8 unknowns**: odd numbers in column $i = 1$, even numbers on the symmetry line $i = 2$.

```
N = round(W / dx)                                   # number of intervals per side

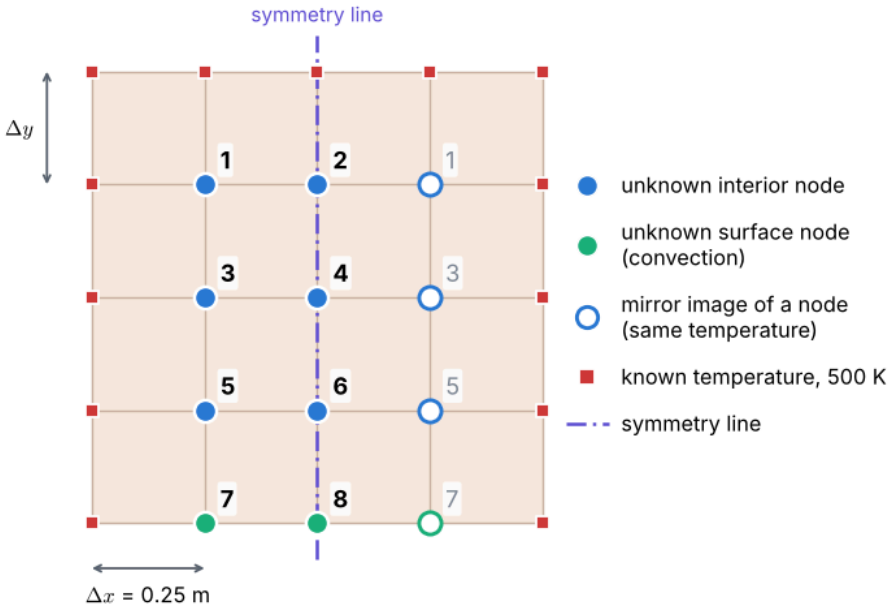
NODES = {1: (1, 3), 2: (2, 3), 3: (1, 2), 4: (2, 2),
         5: (1, 1), 6: (2, 1), 7: (1, 0), 8: (2, 0)}  # node number -> (i, j)
number_of = {ij: n for n, ij in NODES.items()}       # (i, j) -> node number


def mirror(i, j):
    """Use symmetry: column i = 3 has the same temperatures as column i = 1."""
    return (N - i, j) if i > N // 2 else (i, j)


# print the grid row by row, top row first
for j in range(N, -1, -1):
    row = []
    for i in range(N + 1):
        ij = mirror(i, j)
        if ij in number_of:
            name = f"T{number_of[ij]}" + ("'" if ij != (i, j) else " ")
            row.append(f"{name:>6s}")
        else:
            row.append(f"{T_s:6.0f}")
    print(f"j = {j}: " + "".join(row))
print("\nT1' means 'equal to T1 by symmetry'")
```

```
j = 4:    500   500   500   500   500
j = 3:    500   T1    T2    T1'   500
j = 2:    500   T3    T4    T3'   500
j = 1:    500   T5    T6    T5'   500
j = 0:    500   T7    T8    T7'   500

T1' means 'equal to T1 by symmetry'
```



The grid, the eight unknown nodes, their mirror images and the symmetry line.

Step 3 · Physics

Inside the brick: steady 2D conduction without generation, the **Laplace equation**

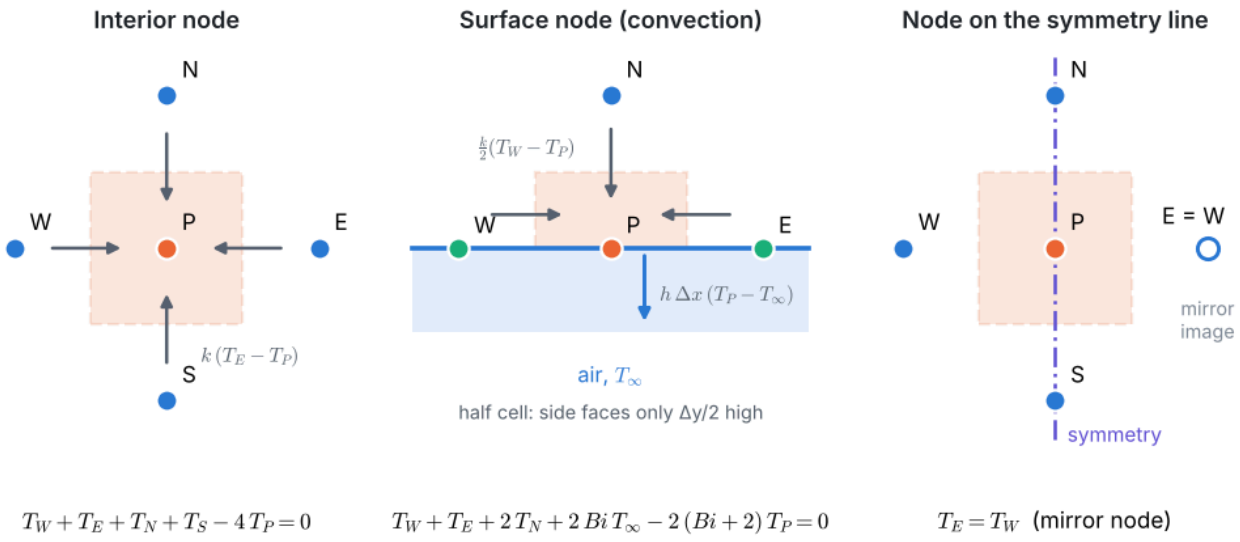
$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0.$$

surface	condition	type
top, left, right	$T = T_s = 500$ K	Dirichlet
bottom ($y = 0$)	$k \dfrac{\partial T}{\partial y} = h\,(T - T_\infty)$	convection (Robin)

The cell **Biot number** compares cooling by the air with conduction through the brick:
$Bi = h\,\Delta x / k = 2.5$. No simple exact solution exists, so the answer is checked by grid refinement.

Step 4 · Numerical scheme: energy balances on control volumes

Around every node, put a small control volume and require that, in steady state, the heat flowing in from all sides adds up to zero. Between two nodes, Fourier's law gives (per unit length of column)
$q = k \times \text{face length} \times (T_{neighbour} - T_P)/\text{distance}$.



Control volumes of the three kinds of node: interior, convective surface (half cell) and symmetry line.

Interior node (full cell, all faces Δx long, $\Delta x = \Delta y$):

$$T_E + T_W + T_N + T_S - 4\,T_P = 0.$$

Surface node with convection (half cell, side faces $\Delta y/2$, top face Δx, bottom face Δx exposed to the air):

$$k\,\frac{\Delta y}{2}\,\frac{T_W - T_P}{\Delta x} + k\,\frac{\Delta y}{2}\,\frac{T_E - T_P}{\Delta x} + k\,\Delta x\,\frac{T_N - T_P}{\Delta y} + h\,\Delta x\,(T_\infty - T_P) = 0$$

$$\Longrightarrow \quad T_W + T_E + 2\,T_N + 2\,Bi\,T_\infty - 2\,(Bi + 2)\,T_P = 0.$$

Node on the symmetry line: its right-hand neighbour is the mirror image of its left-hand one, so that temperature appears twice.

```python
def node_equation(i, j, dx):
    """Energy balance at node (i, j): sum(weight * T_neighbour) + diagonal * T_P + constant = 0."""
    Bi = h * dx / k
    if j == 0:                  # bottom surface: half control volume with convection
        weights = {(i - 1, j): 1.0, (i + 1, j): 1.0, (i, j + 1): 2.0}
        diagonal = -2.0 * (Bi + 2.0)
        constant = 2.0 * Bi * T_inf
    else:                       # interior node: full control volume
        weights = {(i - 1, j): 1.0, (i + 1, j): 1.0, (i, j + 1): 1.0, (i, j - 1): 1.0}
        diagonal = -4.0
        constant = 0.0
    return weights, diagonal, constant
```

The equations are **assembled** into $A\,\mathbf{T} = \mathbf{b}$: an unknown neighbour (after symmetry) adds its weight to A; a known 500 K neighbour and the constant term go to $\mathbf{b}$.

```python
n_unknowns = len(NODES)
A = [[0.0] * n_unknowns for _ in range(n_unknowns)]
b = [0.0] * n_unknowns

for n, (i, j) in NODES.items():
    row = n - 1
    weights, diagonal, constant = node_equation(i, j, dx)
    A[row][row] += diagonal
    b[row] -= constant
    for neighbour, weight in weights.items():
        neighbour = mirror(*neighbour)                # symmetry: column 3 -> column 1
        if neighbour in number_of:                    # unknown: stays on the left
            A[row][number_of[neighbour] - 1] += weight
        else:                                         # known surface at 500 K: to the right
            b[row] -= weight * T_s

print(f"Bi = h dx / k = {h * dx / k}\n")
for row in range(n_unknowns):
    terms = "  ".join(f"{A[row][c]:+g}·T{c + 1}" for c in range(n_unknowns) if A[row][c] != 0)
    print(f"node {row + 1}:  {terms:38s} = {b[row]:7.0f}")
```

```
Bi = h dx / k = 2.5

node 1:  -4·T1  +1·T2  +1·T3                   =   -1000
node 2:  +2·T1  -4·T2  +1·T4                   =    -500
node 3:  +1·T1  -4·T3  +1·T4  +1·T5            =    -500
node 4:  +1·T2  +2·T3  -4·T4  +1·T6            =       0
node 5:  +1·T3  -4·T5  +1·T6  +1·T7            =    -500
node 6:  +1·T4  +2·T5  -4·T6  +1·T8            =       0
node 7:  +2·T5  -9·T7  +1·T8                   =   -2000
node 8:  +2·T6  +2·T7  -9·T8                   =   -1500
```

Step 5 · Solution setup

As in Problem 3: **Gaussian elimination** (direct) and **Gauss–Seidel** (iterative, starting from 400 K everywhere).

```python
def show(Ab, title):
    """Print an augmented matrix [A | b]."""
    print(title)
    for row in Ab:
        print("  [" + "".join(f"{v:8.3f}" for v in row[:-1]) + "  |" + f"{row[-1]:10.2f} ]")
    print()


def gaussian_elimination(A, b, verbose=False):
    """Solve A x = b by forward elimination and back substitution."""
    n = len(b)
    Ab = [A[r][:] + [b[r]] for r in range(n)]          # augmented matrix [A | b]
    if verbose:
        show(Ab, "start:")

    # forward elimination: create zeros below the diagonal, column by column
    for p in range(n - 1):                             # p = pivot row
        for r in range(p + 1, n):                      # every row below the pivot
            factor = Ab[r][p] / Ab[p][p]
            for c in range(p, n + 1):
                Ab[r][c] -= factor * Ab[p][c]
    if verbose:
        show(Ab, "after forward elimination (upper triangular):")

    # back substitution: from the last row up to the first
    x = [0.0] * n
    for r in range(n - 1, -1, -1):
        total = Ab[r][n]
        for c in range(r + 1, n):
            total -= Ab[r][c] * x[c]
        x[r] = total / Ab[r][r]
    return x
```

```python
def gauss_seidel(A, b, T_start, tol=1e-4, max_iter=1000):
    """Solve A x = b by Gauss-Seidel iteration; also return every iterate."""
    T = T_start[:]
    history = [T[:]]                                   # the guess, then every iteration
    changes = []
    for iteration in range(max_iter):
        largest_change = 0.0
        for row in range(len(b)):
            total = b[row]
            for c in range(len(b)):
                if c != row:
                    total -= A[row][c] * T[c]
            new_value = total / A[row][row]
            largest_change = max(largest_change, abs(new_value - T[row]))
            T[row] = new_value                         # use the new value immediately
        history.append(T[:])
        changes.append(largest_change)
        if largest_change < tol:
            break
    return T, history, changes
```

Step 6 · Compute

```python
T = gaussian_elimination(A, b, verbose=True)

for n in NODES:
    print(f"T{n} = {T[n - 1]:.2f} K")
```

```
start:
  [  -4.000   1.000   1.000   0.000   0.000   0.000   0.000   0.000  |  -1000.00 ]
  [   2.000  -4.000   0.000   1.000   0.000   0.000   0.000   0.000  |   -500.00 ]
  [   1.000   0.000  -4.000   1.000   1.000   0.000   0.000   0.000  |   -500.00 ]
  [   0.000   1.000   2.000  -4.000   0.000   1.000   0.000   0.000  |      0.00 ]
  [   0.000   0.000   1.000   0.000  -4.000   1.000   1.000   0.000  |   -500.00 ]
  [   0.000   0.000   0.000   1.000   2.000  -4.000   0.000   1.000  |      0.00 ]
  [   0.000   0.000   0.000   0.000   2.000   0.000  -9.000   1.000  |  -2000.00 ]
  [   0.000   0.000   0.000   0.000   0.000   2.000   2.000  -9.000  |  -1500.00 ]

after forward elimination (upper triangular):
  [  -4.000   1.000   1.000   0.000   0.000   0.000   0.000   0.000  |  -1000.00 ]
  [   0.000  -3.500   0.500   1.000   0.000   0.000   0.000   0.000  |  -1000.00 ]
  [   0.000   0.000  -3.714   1.071   1.000   0.000   0.000   0.000  |   -821.43 ]
  [   0.000   0.000   0.000  -3.096   0.577   1.000   0.000   0.000  |   -759.62 ]
  [   0.000   0.000   0.000   0.000  -3.677   1.093   1.000   0.000  |   -791.93 ]
  [   0.000   0.000   0.000   0.000   0.000  -3.027   0.595   1.000  |   -716.22 ]
  [   0.000   0.000   0.000   0.000   0.000   0.000  -8.339   1.196  |  -2571.43 ]
  [   0.000   0.000   0.000   0.000   0.000   0.000   0.000  -7.996  |  -2711.05 ]

T1 = 489.30 K
T2 = 485.15 K
T3 = 472.07 K
T4 = 462.01 K
T5 = 436.95 K
T6 = 418.74 K
T7 = 356.99 K
T8 = 339.05 K
```

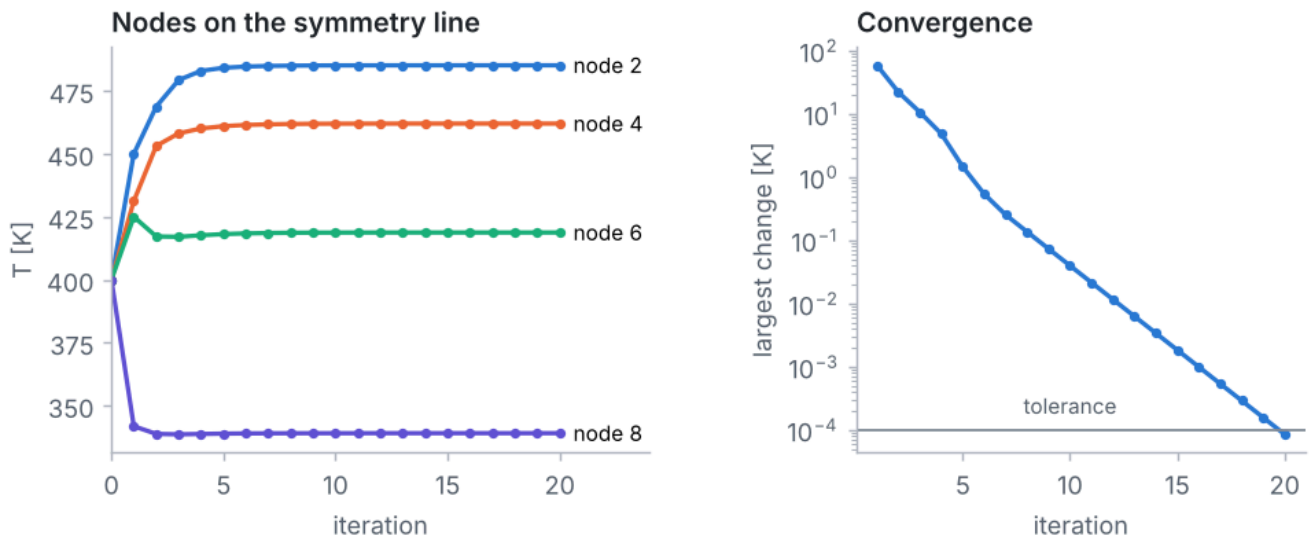
```python
tol = 1e-4
T_gs, history, changes = gauss_seidel(A, b, [400.0] * n_unknowns, tol)

print(f"converged after {len(changes)} iterations\n")
print(" iter " + "".join(f"     T{n}" for n in NODES))
for it in (0, 1, 2, 3, 5, 10, len(changes)):
    print(f"{it:4d}  " + "".join(f"{v:7.2f}" for v in history[it]))
print(f"\nlargest difference from Gaussian elimination: "
      f"{max(abs(p - q) for p, q in zip(T, T_gs)):.1e} K")
```

```
converged after 20 iterations

 iter      T1     T2     T3     T4     T5     T6     T7     T8
   0   400.00 400.00 400.00 400.00 400.00 400.00 400.00 400.00
   1   450.00 450.00 437.50 431.25 434.38 425.00 363.19 341.82
   2   471.88 468.75 459.38 453.12 436.89 417.18 357.29 338.77
   3   482.03 479.30 468.01 458.13 435.62 417.03 356.67 338.60
   5   488.27 484.15 471.08 460.99 436.36 418.12 356.83 338.88
  10   489.26 485.11 472.02 461.96 436.92 418.71 356.99 339.04
  20   489.30 485.15 472.06 462.01 436.95 418.74 356.99 339.05

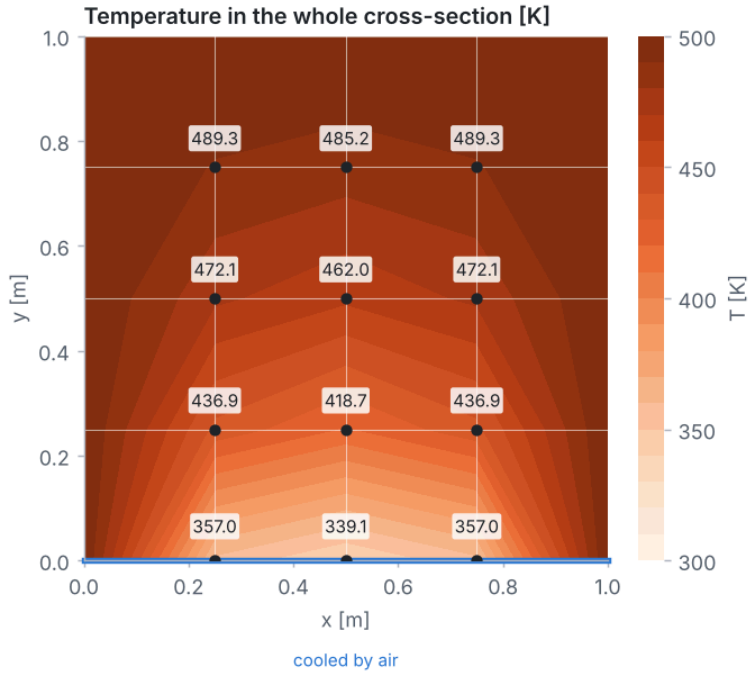
largest difference from Gaussian elimination: 1.0e-04 K
```



Gauss–Seidel: temperatures on the symmetry line (left) and the largest change per iteration (right).

Both methods agree, and agree with the published solution of Incropera's Example 4.3.

Step 7 · Post-processing and verification



Temperature in the whole cross-section, mirrored across the symmetry line.

Heat rate to the air. Each surface node owns a piece of the bottom surface: each corner (part of the hot walls) $\Delta x/2$, node 8 Δx (half on each side of the symmetry line), node 7 and its mirror image Δx each. With Newton's law of cooling and symmetry:

$$q' = 2h\left[\frac{\Delta x}{2}(T_s - T_\infty) + \Delta x\,(T_7 - T_\infty) + \frac{\Delta x}{2}(T_8 - T_\infty)\right].$$

```
T7, T8 = T[6], T[7]
q_corners = 2 * h * (dx / 2) * (T_s - T_inf)
q_nodes = 2 * h * (dx * (T7 - T_inf) + (dx / 2) * (T8 - T_inf))
q_air = q_corners + q_nodes

print(f"from the two corners (at 500 K): {q_corners:6.1f} W/m")
print(f"from nodes 7, 8 and 7':          {q_nodes:6.1f} W/m")
print(f"heat rate to the air:     q' = {q_air:6.1f} W/m")
```

```
from the two corners (at 500 K):  500.0 W/m
from nodes 7, 8 and 7':           382.6 W/m
heat rate to the air:     q' =  882.6 W/m
```

Energy check. The heat conducted from all known 500 K nodes into their unknown neighbours (with the same face lengths as the energy balances) must equal the heat convected from the unknown surface nodes:

```
def heat_flows(Tf, N):
    """Heat [W/m] conducted in from the hot walls and convected out by the unknown surface
nodes."""
    dx = W / N
    unknown = lambda i, j: 0 < i < N and 0 <= j < N             # noqa: E731
    q_in = 0.0
    for j in range(N):
        for i in range(1, N):                                    # every unknown node P
            for ni, nj in ((i - 1, j), (i + 1, j), (i, j + 1), (i, j - 1)):
                if nj >= 0 and not unknown(ni, nj):              # a known 500 K neighbour
                    face = dx / 2 if (j == 0 and nj == 0) else dx
                    q_in += k * face * (Tf[nj][ni] - Tf[j][i]) / dx
    q_out = sum(h * dx * (Tf[0][i] - T_inf) for i in range(1, N))
    return q_in, q_out


# the full 5 x 5 field Tf[j][i] (row j = 0 is the bottom), with the mirror images
Tf = [[T_s] * (N + 1) for _ in range(N + 1)]
for n, (i, j) in NODES.items():
    Tf[j][i] = Tf[j][N - i] = T[n - 1]

q_in, q_out = heat_flows(Tf, N)
print(f"conducted in from the 500 K walls:   {q_in:7.2f} W/m")
print(f"convected out by nodes 7, 8 and 7':  {q_out:7.2f} W/m")
print(f"difference:                          {abs(q_in - q_out):7.2f} W/m")
```

```
conducted in from the 500 K walls:    382.60 W/m
convected out by nodes 7, 8 and 7':   382.60 W/m
difference:                             0.00 W/m
```

The two are equal: the control-volume balances add up to a balance for the whole column, so **the discrete solution conserves energy**. The remaining 500 W/m reach the air through the two corners, which are part of the hot walls.

Grid refinement. A general SOR solver for the full cross-section (no symmetry needed) solves the same problem on finer grids:

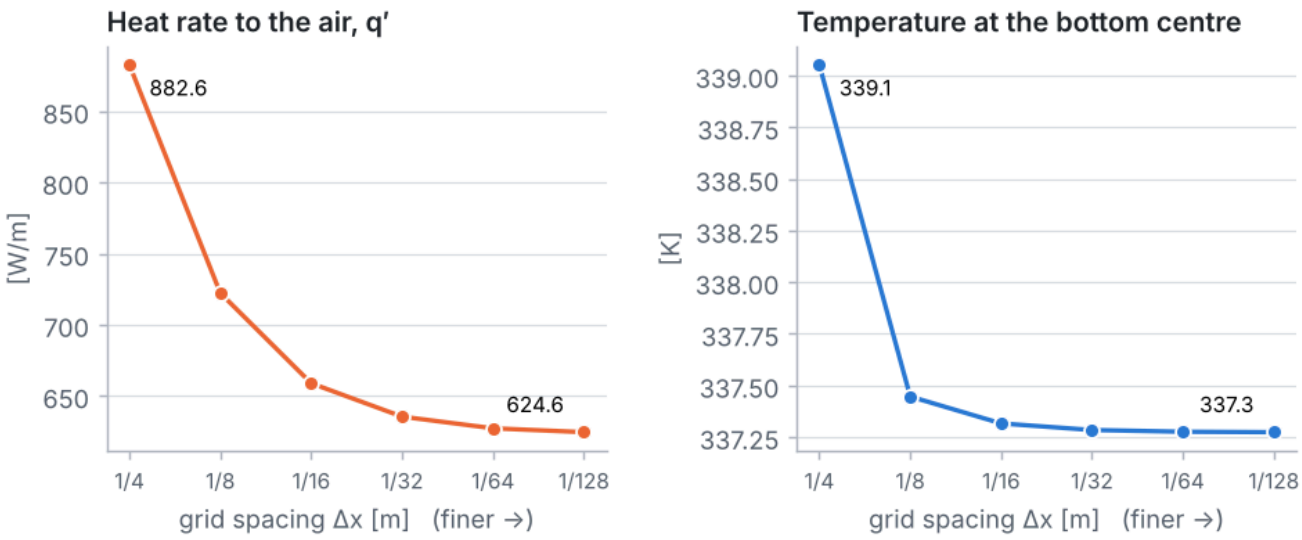
```python
def solve_column(N, tol=1e-6):
    """Temperatures Tf[j][i] in the column on a grid with N x N intervals, by SOR."""
    dx = W / N
    Bi = h * dx / k
    omega = 2.0 / (1.0 + math.sin(math.pi / N))         # over-relaxation factor
    Tf = [[T_s] * (N + 1) for _ in range(N + 1)]        # walls and top at 500 K ...
    for j in range(N):
        for i in range(1, N):
            Tf[j][i] = 400.0                             # ... and a guess for the unknowns

    for sweep in range(1, 100_000):
        largest_change = 0.0
        for j in range(N):
            for i in range(1, N):
                if j == 0:                               # surface node with convection
                    new = (Tf[0][i - 1] + Tf[0][i + 1] + 2 * Tf[1][i] + 2 * Bi * T_inf) / (2 * (Bi
+ 2))
                else:                                    # interior node
                    new = (Tf[j][i - 1] + Tf[j][i + 1] + Tf[j + 1][i] + Tf[j - 1][i]) / 4
                new = Tf[j][i] + omega * (new - Tf[j][i])
                largest_change = max(largest_change, abs(new - Tf[j][i]))
                Tf[j][i] = new
        if largest_change < tol:
            break
    return Tf, sweep


def heat_to_air(Tf, N):
    """q' [W/m]: the two corners (half a cell each, at 500 K) plus every unknown surface node."""
    dx = W / N
    return 2 * h * (dx / 2) * (T_s - T_inf) + sum(h * dx * (Tf[0][i] - T_inf) for i in range(1,
N))
```

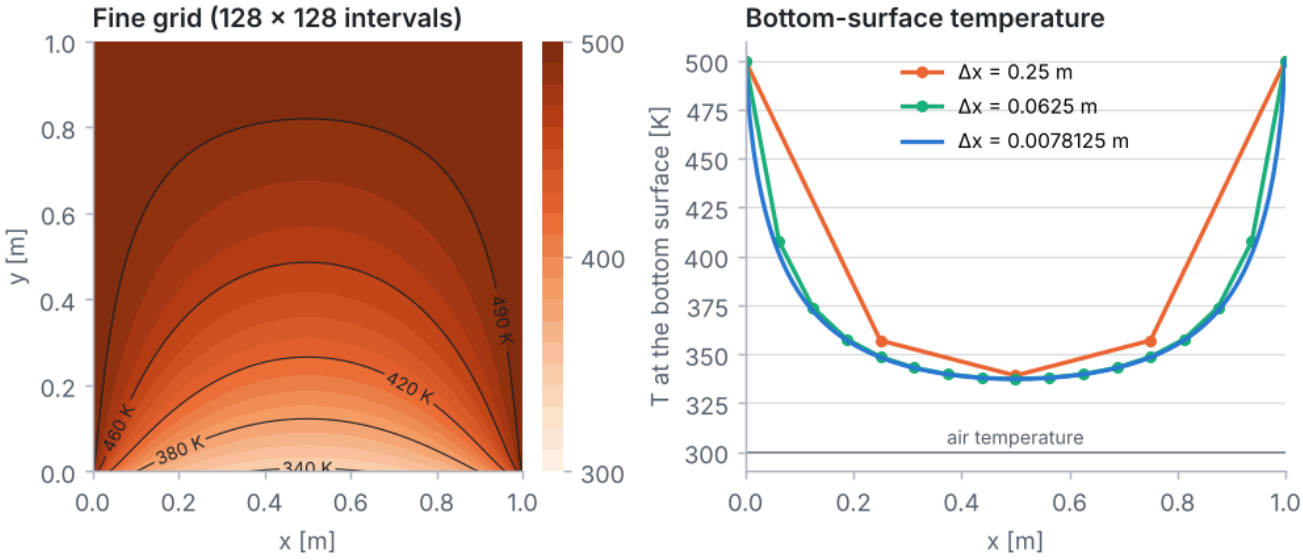
```python
grids = [4, 8, 16, 32, 64, 128]
results = {}
print("   N   dx [m]   sweeps   T bottom centre [K]   q' [W/m]   of which corners   in - out")
for n_int in grids:
    Tf_n, sweeps = solve_column(n_int)
    results[n_int] = Tf_n
    q_in_n, q_out_n = heat_flows(Tf_n, n_int)
    corners = 2 * h * (W / n_int / 2) * (T_s - T_inf)
    print(f"{n_int:4d}  {W / n_int:7.4f}  {sweeps:6d}   {Tf_n[0][n_int // 2]:14.2f}        "
          f"{heat_to_air(Tf_n, n_int):8.1f}   {corners:12.1f}      {q_in_n - q_out_n:+.1e}")
```

```
   N   dx [m]   sweeps   T bottom centre [K]   q' [W/m]   of which corners   in - out
   4   0.2500      18           339.05           882.6          500.0      +1.9e-06
   8   0.1250      36           337.45           722.3          250.0      +8.8e-06
  16   0.0625      70           337.32           659.0          125.0      +2.4e-05
  32   0.0312     136           337.28           635.5           62.5      +5.0e-05
  64   0.0156     261           337.28           627.3           31.2      +1.1e-04
 128   0.0078     509           337.27           624.6           15.6      +1.8e-04
```



Heat rate to the air and bottom-centre temperature as the grid is refined.

Temperatures converge fast; the heat rate does not. The bottom-centre temperature only moves from 339.1 K to 337.3 K, but the heat rate drops from **883 W/m** to about **625 W/m**: the coarse grid overestimates the heat loss by about 40 %. On the coarse grid each bottom corner, held at 500 K, is credited with 0.125 m of hot surface exposed to the air, which alone gives 500 W/m. On finer grids this strip shrinks and the surface next to the corner cools quickly. Steep gradients near corners need fine grids, which is why real CFD meshes are refined near corners and walls.



Left: the temperature field on a 128×128 grid. Right: the bottom-surface temperature on three grids; only fine grids resolve the steep drop next to the corners.

Result. On the 0.25 m grid: $T_1 = 489.30$, $T_2 = 485.15$, $T_3 = 472.07$, $T_4 = 462.01$, $T_5 = 436.95$, $T_6 = 418.74$, $T_7 = 356.99$, $T_8 = 339.05$ K, and the heat rate to the air is $q' \approx$ **883 W/m**, as in Incropera's example. Grid refinement shows that the temperatures are already accurate to within about 2 K, while the heat rate converges to about **625 W/m**.

Key ideas

- Energy balances on control volumes give the node equations, and handle convection boundaries naturally.
- A convective surface node uses a half control volume:
  $T_W + T_E + 2T_N + 2\,Bi\,T_\infty - 2(Bi + 2)T_P = 0$.
- Symmetry halves the number of unknowns; a node on the symmetry line counts its mirror neighbour twice.
- The finite difference solution conserves energy exactly.
- A grid that is fine enough for temperatures may be far too coarse for heat fluxes near corners: always check with grid refinement.